

ShellForge: A Unix Shell in C, from System Calls to POSIX Threads

*A Project Based Learning Report Submitted in partial fulfilment of the requirements for the
award of the degree*

of

Bachelor of Technology

in The Department of Artificial Intelligence and Data Science

Operating Systems and Systems Programming (25CS2104E)

Submitted by

2510080006: M. Sathya Krishna

2510080001: Yashwanth

2510080005: Aneeq

2510080002: Abhinav

Under the guidance of

Dr. P. Kalpana



Department of Artificial Intelligence and Data Science

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

OCT - 2026.

Abstract

A shell is the program through which a user reaches the operating system, and writing one touches nearly every service the kernel offers: creating and replacing processes, waiting for them, delivering signals, connecting programs with pipes and redirecting their files. Production shells hide these mechanisms under hundreds of thousands of lines, and short textbook examples skip the hazards between them. This project builds a shell directly on POSIX system calls, one course chapter at a time, and proves each step.

ShellForge is 1,357 lines of C in nine modules, developed over ten weekly chapters of the course handbook: a read-eval-print loop, dynamic input, a parser, fork, exec and wait, built-in commands, signal handling, pipes and inter-process communication, memory debugging, input and output redirection, and POSIX threads with a background monitor thread and a race-condition demonstration. The shell is also part of the Enterprise Knowledge Intelligence Platform: the backend's container image compiles it, and a web page runs it on the server in a sandbox, on preset sessions only.

Seven automated suites make 180 checks, all passing locally and in continuous integration. Valgrind reports that all 31 heap blocks of a full session, threads included, are freed, with no errors, and ThreadSanitizer locates the deliberate race in a single line. Without a lock, threads lose 5% of their increments with two threads and 71% with sixteen; under a mutex none are lost, at 12 to 15 times the cost. Two hazards in the handbook's own listings were found and fixed, the second one only exposed by adding a thread: with it, 270 to 280 of 300 commands lost their exit status. Mutation tests confirm that the checks catch each fix being undone.

Keywords: Unix shell, system calls, fork and exec, waitpid, signals, pipes, file descriptors, I/O redirection, POSIX threads, mutex, race condition, Valgrind, ThreadSanitizer

List of Figures

Figure 1: ShellForge's modules and how a command line flows through them.....	7
Figure 2: The ten weekly chapters and the course outcome each serves	8
Figure 3: One external command: the shell and its child, with SIGCHLD blocked around the wait	9
Figure 4: Descriptors for <code>ls wc -l</code> after each child's <code>dup2()</code>	9
Figure 5: Two increments of a shared counter, without a lock and under a mutex	10
Figure 6: Where SIGCHLD goes once the shell has a second thread, and the fix.....	11
Figure 7: From the ShellForge page to the shell and back	11
Figure 8: make test: every suite passing in the gcc 13 container with Valgrind and GDB.....	13
Figure 9: A ShellForge session: built-ins, a pipe, <code>></code> , <code>>></code> , <code><</code> , <code>2></code> and an unknown command.....	13
Figure 10: demo-ipc: parent-to-child and child-to-parent messages, and end-of-file.....	14
Figure 11: Valgrind: no leaks and no errors; GDB inspecting the argument vector.....	14
Figure 12: demo-threads and the monitor thread; the shell's threads seen through <code>/proc</code>	15
Figure 13: ThreadSanitizer locating the race, and the signal-mask mutation	15
Figure 14: A redirected command's descriptors: correct build, and with <code>F_DUPFD</code>	16
Figure 15: The race lab on the ShellForge page	17
Figure 16: Preset sessions, and the transcript of the redirection session	17

List of Tables

Table 1: Course outcomes, the weeks that cover them and their system calls	7
Table 2: Redirection operators and how ShellForge opens their files	9
Table 3: ShellForge sessions on the web page.....	12
Table 4: ShellForge test suites	13
Table 5: Updates lost by an unprotected counter (100,000 increments per thread, five runs each).....	15
Table 6: Mutation tests: undoing a fix must make its check fail.....	16
Table 7: Challenges and how they were solved.....	18

Table of Contents

1. Introduction	6
2. Methodology	7
a. Requirements and System Design	7
b. Development Method	8
c. Input and Parsing (Weeks 1–3)	8
d. Processes, Built-ins and Signals (Weeks 4–6)	8
e. Pipes, IPC and Redirection (Weeks 7 and 9)	9
f. Memory Debugging (Week 8)	10
g. Threads and Concurrency (Week 10)	10
h. Integration with the Web Platform	11
3. Experiments	12
4. Results	12
a. Correctness	12
b. The Shell at Work	13
c. Memory	14
d. Concurrency	14
e. Mutation Tests	15
f. The Web Page	16
g. Challenges	17
5. Conclusion and Future Work	18
References	18

ShellForge: A Unix Shell in C, from System Calls to POSIX Threads

1. INTRODUCTION

An operating system offers its services through a narrow interface of system calls, and the shell is the most familiar program built on top of it [1], [6], [7]. Typing `ls | wc -l > count.txt` asks the shell to read and parse a line, create two processes, connect them through a kernel pipe, point one at a file, replace both with other programs, and wait for them, all while staying alive when the user presses Ctrl+C. Each step is one of the course's topics: processes, signals, inter-process communication, memory and file descriptors, and, since the shell can run work in the background, threads.

Existing shells are poor teaching material for the same reason they are good tools. Bash and its relatives hide the mechanisms under decades of features, and textbook examples show each system call on its own [2], [3]. The difficult part of a shell is the interaction between them. A signal handler that calls `printf()` is undefined behaviour. A reaper for finished children can collect a command before the shell's own `waitpid()` does. A descriptor saved during a redirection leaks into every program the shell starts. A thread added for one feature silently changes where signals are delivered. The course handbook's own listings contain several of these, which made them the most instructive part of the project.

Problem statement. Build a working Unix shell in C directly on POSIX system calls [9], following the course handbook week by week, so that every operating-system concept in the syllabus is implemented, tested automatically and checked for memory and concurrency errors, and make it usable from the project's web platform without giving web users a way to run arbitrary commands on the server.

Approach. Each chapter is implemented as its own module, tested by a self-checking suite, run under AddressSanitizer, Valgrind and, for threads, ThreadSanitizer, and documented with every place where the code deliberately differs from the handbook and why. Hazards are not argued away: each fix is backed by a test that fails when the fix is removed.

Objectives.

- Implement a shell with an interactive loop, unbounded input, a parser, process execution, built-ins, signals, two-stage pipelines and four redirection operators.
- Add POSIX threads: a background monitor thread, joined at exit, and a race-condition demonstration with and without a mutex.
- Keep every allocation freed and every descriptor closed, verified with Valgrind, sanitizers and `/proc`.
- Test every chapter automatically, in a container locally and in GitHub Actions on every change.
- Map the work to the course outcomes CO1–CO6.
- Integrate the shell into the web platform safely, as a sandboxed set of preset sessions.

Scope. The project covers the ten chapters published so far, from the read-eval-print loop to threads. Job control, named pipes, Unix domain sockets, shared memory, semaphores and `mmap` remain for later chapters. Section 2 describes the design and each part of the shell, Section 3 the testing method, Section 4 the results and Section 5 concludes.

2. METHODOLOGY

a. Requirements and System Design

The requirements come from the course outcomes. Each outcome maps to a group of system calls and to the weeks that use them (Table 1).

Table 1: Course outcomes, the weeks that cover them and their system calls

Course outcome	Weeks	System calls and library functions
CO1: the shell as a user-space program over the system-call interface	1–3	read loop over <code>getchar()</code> ; <code>malloc</code> , <code>realloc</code> , <code>free</code> ; in-place tokenising
CO2: process control	4–6	<code>fork</code> , <code>execvp</code> , <code>waitpid</code> , <code>_exit</code> , <code>chdir</code> , <code>getcwd</code> , <code>setenv</code>
CO3: inter-process communication and signals	6–7	<code>sigaction</code> , <code>sigprocmask</code> , <code>kill</code> ; <code>pipe</code> , <code>dup2</code> , <code>close</code>
CO4: memory	2, 8	heap growth; Valgrind, GDB, AddressSanitizer
CO5: files and descriptors	9	<code>open</code> with <code>O_CREAT</code> , <code>O_TRUNC</code> , <code>O_APPEND</code> ; <code>dup2</code> ; <code>fcntl(F_DUPFD_CLOEXEC)</code>
CO6: threads and synchronisation	10	<code>pthread_create</code> , <code>pthread_join</code> , <code>pthread_mutex_lock</code> , <code>pthread_cond_timedwait</code> , <code>pthread_sigmask</code>

ShellForge is a loop: `main.c` prints a prompt, `input.c` reads a line of any length, `parser.c` splits it into a NULL-terminated argument vector, and the number of pipe symbols decides what runs (Figure 1). Without a pipe, `redirect.c` applies any redirections in the shell itself, then a built-in runs in the shell process or `executor.c` forks a child. With one pipe, `pipes.c` builds a two-stage pipeline. `signals.c` and `thread.c` work across all of them.

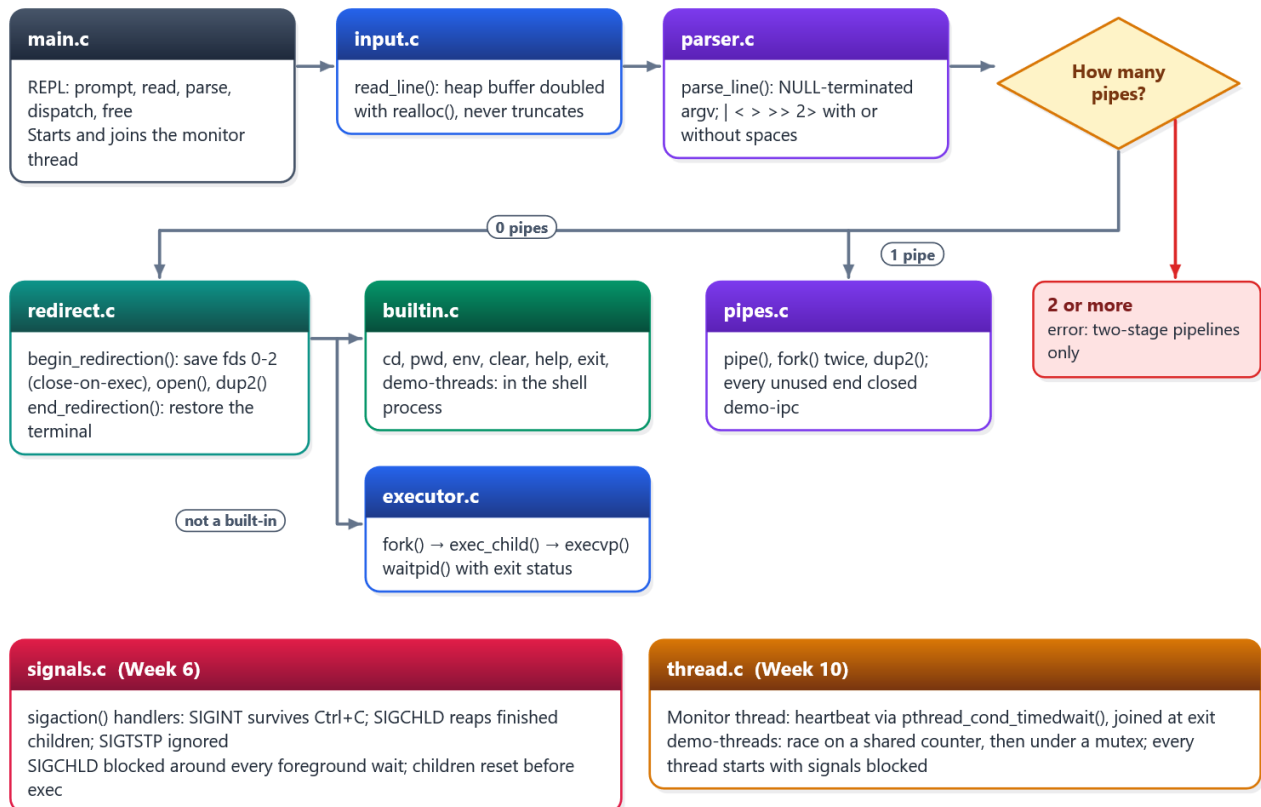
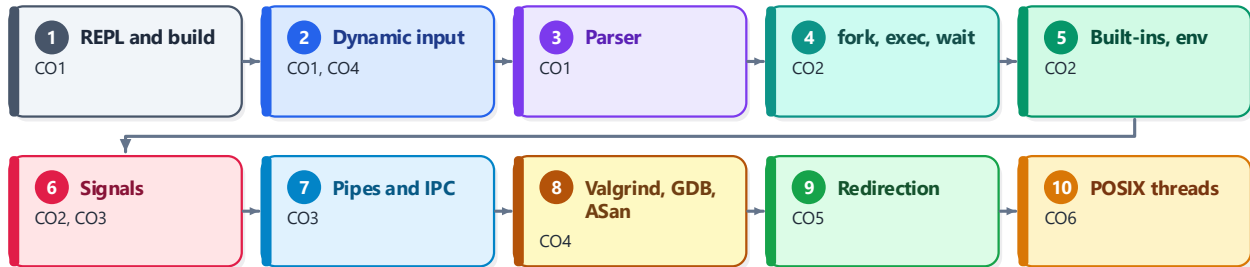


Figure 1: ShellForge's modules and how a command line flows through them

b. Development Method

Each week followed the same cycle (Figure 2): implement the handbook chapter as a module, add an automated suite that drives the shell through its standard input, rebuild under sanitizers and Valgrind, and write the week's document, including every deliberate difference from the handbook. Only then was the week committed. Earlier suites run again every time, so a later week cannot quietly break an earlier one.



Each week: implement the chapter → automated suite → sanitizers and Valgrind → week document with differences from the handbook → commit

Figure 2: The ten weekly chapters and the course outcome each serves

c. Input and Parsing (Weeks 1–3)

`read_line()` starts with a 64-byte heap buffer and doubles it with `realloc()` whenever it fills, so a 3,000-character line arrives intact, and every allocation result is checked. `parse_line()` splits the line in place on spaces, tabs and line ends into a vector that grows from 64 to 128 to 256 entries, ending in `NULL` exactly as `execvp()` expects. From Week 9 it also recognises `|`, `<`, `>`, `>>` and `2>` with or without surrounding spaces. Because the operator character in the line is overwritten to end the word before it, operators are returned as static strings.

The handbook's Week 3 listing does not compile as printed: its `#include` lines carry a stray quote, and its Makefile target has no prerequisites, so it never rebuilds. ShellForge keeps an object-file Makefile with `-Wall -Wextra -g` and builds with no warnings.

d. Processes, Built-ins and Signals (Weeks 4–6)

An external command runs in a child (Figure 3). The shell flushes its output buffers, because a child inherits them and would print them a second time, then calls `fork()`. The child restores default signal handling, applies redirections and calls `execvp()`. If that fails it reports the error and leaves with `_exit()`, not `exit()`, so it does not flush the copied buffers or run exit handlers. The parent waits with `waitpid()` and decodes the result with `WIFEXITED`, `WEXITSTATUS` and `WIFSIGNALED`.

Built-ins (`cd`, `pwd`, `env`, `clear`, `help`, `exit`) run inside the shell process, because a child's `chdir()` would vanish with the child. `cd` keeps `PWD` in step, since the kernel tracks the working directory but `PWD` is just an environment variable. `exit` returns to `main()` instead of calling `exit()`, so the line buffer and token vector are freed first.

Week 6 installs handlers with `sigaction()` and `SA_RESTART`, so `Ctrl+C` at the prompt does not appear as end-of-input. The terminal sends `Ctrl+C` to the whole foreground process group: the child dies with the default action and the shell carries on. A `SIGCHLD` handler reaps finished children with `waitpid(-1, NULL, WNOHANG)` in a loop, because signals do not queue. Handlers use `write()`, since `printf()` is not async-signal-safe [2], [3].

A race in the handbook, fixed. The handbook's reaper can collect the foreground child before the shell's own `waitpid()` does, and its exit status is lost. ShellForge blocks `SIGCHLD` with `sigprocmask()` from just before `fork()` until its `waitpid()` returns. A `SIGCHLD` that arrives meanwhile is delivered as soon as it is unblocked, so other children are still reaped.

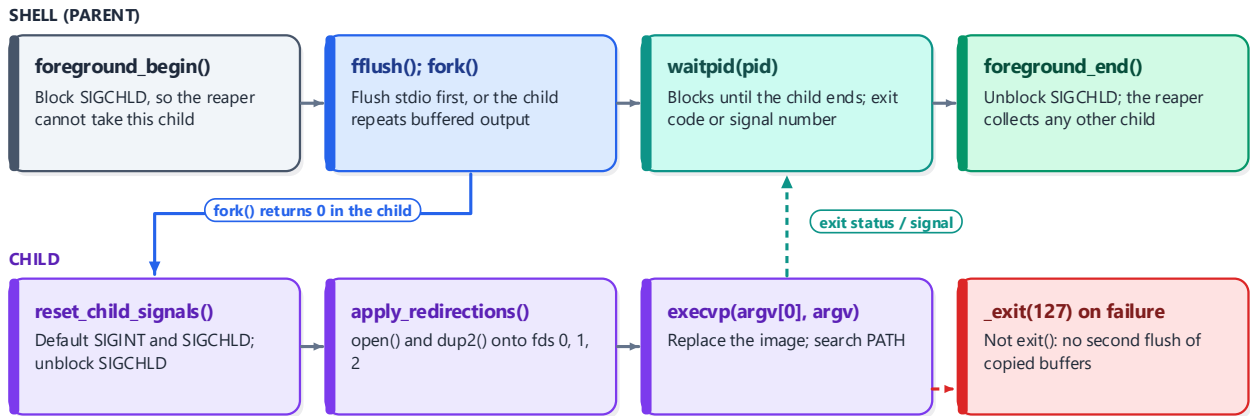


Figure 3: One external command: the shell and its child, with SIGCHLD blocked around the wait

e. *Pipes, IPC and Redirection (Weeks 7 and 9)*

For `cmd1 | cmd2` the shell creates a pipe, forks twice, and each child moves its end onto standard output or input with `dup2()` before calling `execvp()` (Figure 4). Every unused copy of each end must be closed, in both children and in the parent. As long as any write end stays open, the reader never sees end-of-file and the pipeline hangs. A `demo-ipc` command shows messages passed from parent to child and back, and a `read()` returning 0 once the last write end is closed.

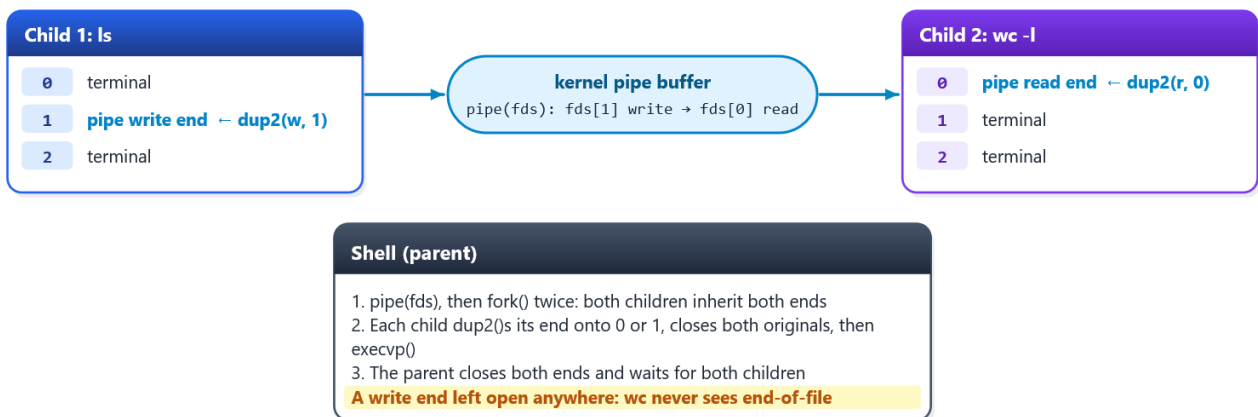


Figure 4: Descriptors for `ls | wc -l` after each child's `dup2()`

Redirection points a standard stream at a file (Table 2). Programs only ever write to descriptor 1, so after `dup2(fd, 1)` their output goes to the file without their knowledge. For built-ins such as `pwd > file`, the shell itself saves descriptors 0–2, redirects, runs the built-in and restores the terminal. The saved copies are made with `fcntl(F_DUPFD_CLOEXEC)`, so they close automatically in any program the shell starts. Otherwise they leak into children, a mistake the tests detect (Section 4.5).

Table 2: Redirection operators and how ShellForge opens their files

Operator	Stream	open() flags
> file	standard output	O_WRONLY O_CREAT O_TRUNC, mode 0644
>> file	standard output	O_WRONLY O_CREAT O_APPEND, mode 0644
< file	standard input	O_RDONLY
2> file	standard error	O_WRONLY O_CREAT O_TRUNC, mode 0644

The handbook's listing implements only `>` and `>>`, forks a child of its own and runs after the built-in check, so `pwd > file` would still print to the terminal. ShellForge implements all four operators, strips them from the vector so the Week 4 executor runs the command unchanged, covers built-ins and pipelines, and reports `echo hi >` as a syntax error rather than calling `open(NULL)`.

f. Memory Debugging (Week 8)

Week 8 adds the tools rather than a feature. `make asan` rebuilds with AddressSanitizer and UndefinedBehaviorSanitizer [12], and `make valgrind` runs a session under Valgrind's Memcheck [11]. A scripted GDB session stops in the parser and inspects the argument vector. Every suite reruns under the sanitizers. Each tool is first shown catching a deliberate error, a 200-byte leak or a heap overflow, so a clean report means something. A review that week fixed three latent bugs: `errno` was read after `fprintf()` could change it, a pipeline read a status `waitpid()` might not have set, and three copies of the child set-up code were merged into one `exec_child()`.

g. Threads and Concurrency (Week 10)

A thread runs inside its process and shares the process's code, globals and heap, with its own stack and registers [7], [10]. Week 10 adds two uses of threads, both in `thread.c`.

The monitor thread. Started after the signal handlers are installed, it prints `[Monitor] ShellForge Running...` every `SHELLFORGE_MONITOR` seconds (10 by default, 0 for off), while the main thread keeps reading and running commands. The handbook's version calls `sleep(10)` in an endless loop and is detached, so nothing can stop it: the process exits with it mid-sleep, and Valgrind reports the 272 bytes of thread storage that `pthread_create()` allocated as possibly lost. ShellForge's monitor instead waits on a condition variable with a deadline, so `stop_monitor_thread()` wakes it at once and joins it with `pthread_join()` before `main()` returns.

The race and the mutex. The demo-threads `[threads] [increments]` built-in runs the handbook's experiment: each worker adds 1 to a shared counter, first with no lock and then under a mutex, and every worker is joined before the total is read. `counter++` is a load, an add and a store, so two threads can load the same value and one increment is lost (Figure 5). Under the mutex the increment is a critical section that one thread enters at a time [10].

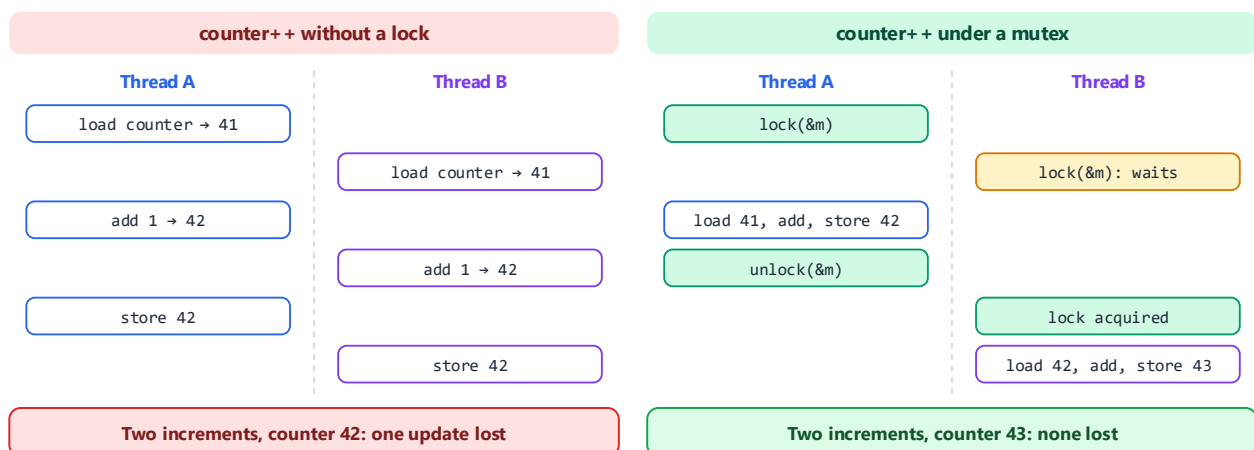


Figure 5: Two increments of a shared counter, without a lock and under a mutex

Threads and signals, a second hazard. Adding the monitor broke Week 6. `sigprocmask()` changes only the calling thread's mask, and the kernel delivers a process-directed signal to any thread that does not block it [2], [10]. While the main thread waited for a command with `SIGCHLD` blocked, the signal went to the monitor. The reaper then ran there, collected the child, and the

shell's `waitpid()` failed with `ECHILD` (Figure 6). `thread.c` therefore creates every thread with all signals blocked. It fills the mask with `pthread_sigmask()` around `pthread_create()`, because a new thread inherits its creator's mask, then restores it.

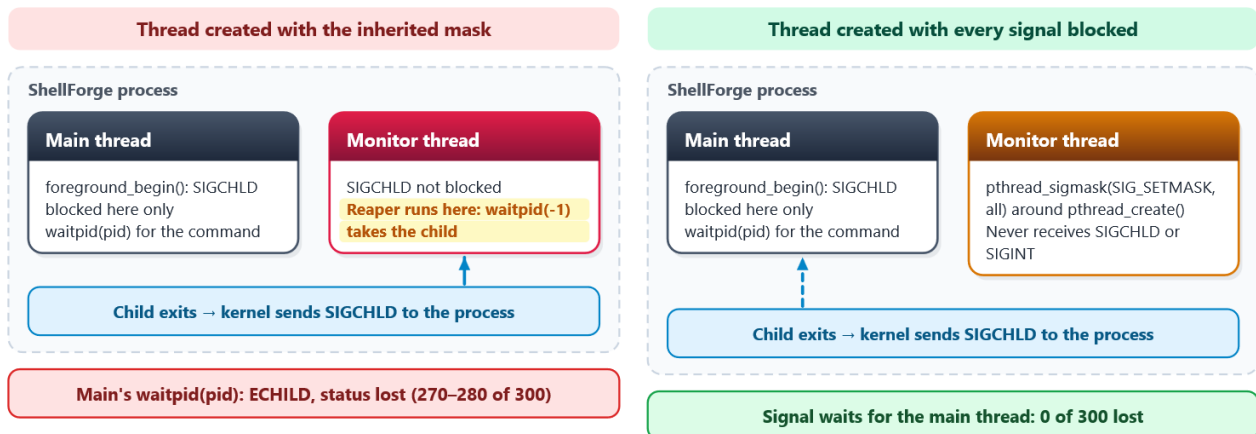


Figure 6: Where SIGCHLD goes once the shell has a second thread, and the fix

h. Integration with the Web Platform

The project's architecture placed the shell behind the platform's backend. The backend's Dockerfile compiles ShellForge in a build stage on the runtime image's own Ubuntu base, so the binary matches the runtime's C library, and copies it into the image [15]. ShellForgeService starts it with Java's `ProcessBuilder`, itself a fork and exec, and the ShellForge page shows the result (Figure 7, Table 3).

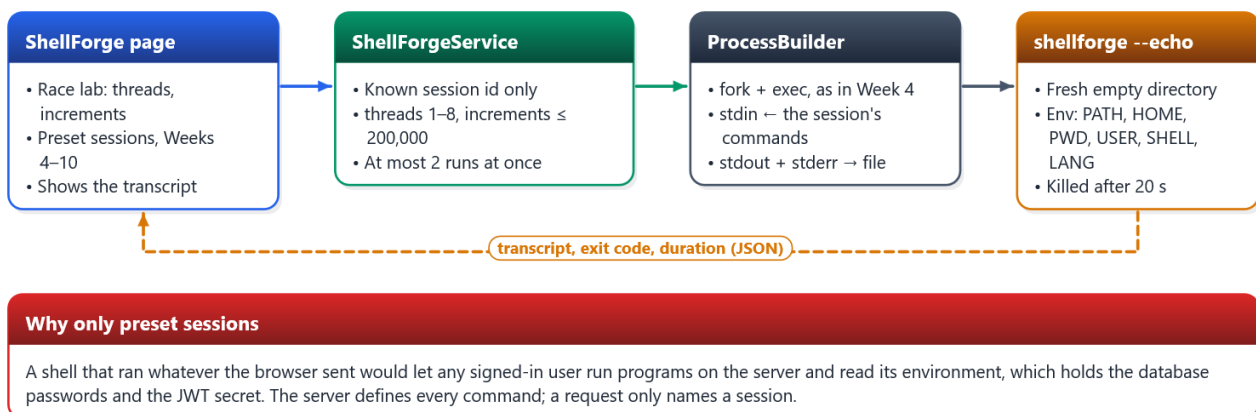


Figure 7: From the ShellForge page to the shell and back

A shell that ran whatever the browser sent would let any signed-in user run programs on the server and read its environment, which holds the database passwords and the token-signing secret. So the server defines every command, and a request only names a session. Each run starts the shell with `--echo`, which prints each command after its prompt so the output reads as a transcript. It runs in a new empty directory with an environment of only `PATH`, `HOME`, `PWD`, `USER`, `SHELL` and `LANG`, under a 20-second limit and a 64 KB output cap, and at most two runs execute at once on the 0.1-CPU instance.

Table 3: ShellForge sessions on the web page

Session	Week	Commands
Processes	4	echo, ls /usr, a missing program, echo
Built-ins	5	pwd, cd /usr/bin, cd, env, help
Pipes and IPC	7	echo ... tr a-z A-Z, ls /usr/bin wc -l, demo-ipc
Redirection	9	>, >>, <, sort -r < notes.txt > sorted.txt, 2>, ls
Monitor thread	10	sleep 2.5 with a 1-second heartbeat
Race lab	10	demo-threads with 1–8 threads and up to 200,000 increments each

3. EXPERIMENTS

The development machine runs Windows without a C toolchain, so ShellForge is built and tested in a gcc:13 container, extended with Valgrind and GDB for Week 8. The evidence in Section 4 comes from one run of that container on an AMD Ryzen 7 260 with 16 logical CPUs (Linux 6.6, gcc 13.4, Valgrind 3.19, GDB 13.1). The same suites run in GitHub Actions on Ubuntu 24.04 before the backend tests, on every change. The suites drive the real binary through its standard input and inspect what it prints, the files it creates and the kernel’s view of it, using five techniques:

- **Scripted sessions.** Each case pipes a few lines into the shell and checks the output and the files left behind, for example `> truncates` or `cd` persisting into the next `pwd`.
- **Real signals.** For Ctrl+C and Ctrl+Z the shell runs in its own process group under `setsid`, and the test signals the whole group the way a terminal does. A Ctrl+C during `sleep 5` must end the sleep but not the shell.
- **The kernel’s view through `/proc`.** No zombie children (state Z) while the shell idles; one entry in `/proc/<pid>/task` per thread; only inherited descriptors visible in a child.
- **Tools with controls.** AddressSanitizer, UndefinedBehaviorSanitizer and Valgrind run over complete sessions, and ThreadSanitizer [13] over the thread demo. Each is first shown catching a planted error.
- **Mutation tests [14].** A fix is undone on purpose: the signal mask removed from new threads, or `F_DUPFD` used instead of `F_DUPFD_CLOEXEC`. The check that guards it must then fail.

The race was measured separately: `demo-threads` with 1, 2, 4, 8 and 16 threads and 100,000 increments each, five runs per setting, recording the unprotected total, the protected total and both times. On the platform side, `ShellForgeServiceTest` runs the real binary for every session in CI, checks the sandbox environment, and kills a deliberately runaway session at its time limit. Integration tests check that unknown sessions and out-of-range numbers are rejected and that the endpoints require sign-in.

4. RESULTS

a. Correctness

All seven suites pass: 180 checks with none failed or skipped (Table 4, Figure 8). The C code builds with no warnings under `-Wall -Wextra`. ShellForge is 1,357 lines of C in nine modules; the tests are 1,075 lines.

Table 4: ShellForge test suites

Suite	Checks	What it covers
test_week4.sh	21	running programs, command not found, non-zero exits, a 1,500-character line, 200 arguments, --debug-tokens
test_week5.sh	32	cd persisting in the parent, \$HOME, PWD, env with unset variables, help, clear
test_week6.sh	21	Ctrl+C on a command, a pipeline and the prompt; Ctrl+Z; signal state in the child; zombies reaped; no stolen status
test_week7.sh	23	the IPC demo and end-of-file, two-stage and unspaced pipelines, syntax errors, long arguments
test_week8.sh	22	Valgrind leak-free on two exit paths, GDB session, ASan, planted-error controls
test_week9.sh	29	all four operators, combined and unspaced, built-ins, pipelines, no descriptor leaks
test_week10.sh	32	monitor heartbeat and join, threads in /proc, no stolen exit status, race and mutex, TSan
Total	180	Every suite also reruns under ASan and UBSan with no report

```

make test: Weeks 4-10 (gcc 13, Valgrind 3.19, GDB 13)

$ make test
Week 4  processes and command execution      21 passed, 0 failed
Week 5  built-ins and environment variables  32 passed, 0 failed
Week 6  signals and process control         21 passed, 0 failed
Week 7  pipes and IPC                      23 passed, 0 failed
Week 8  Valgrind, GDB and AddressSanitizer  22 passed, 0 failed, 0 skipped
Week 9  file descriptors and redirection    29 passed, 0 failed
Week 10 threads and concurrency            32 passed, 0 failed, 0 skipped

Total: 180 checks, 0 failed, 0 skipped
$ echo $?
0

```

Figure 8: make test: every suite passing in the gcc 13 container with Valgrind and GDB

b. The Shell at Work

Figure 9 shows built-ins, a pipeline, all three file redirections and an unknown command in one session. `cd` changes the shell's own directory, `2>` sends the error into a file, and the shell reports the missing program and carries on. Figure 10 shows the IPC demonstration: messages in both directions, and `read()` returning 0 once the write end is closed.

```

ShellForge 10.0: built-ins, pipes, redirection, errors

=====
Welcome to ShellForge Version 10.0
=====

myshell> pwd
/tmp/demo
myshell> cd /usr
myshell> pwd
/usr
myshell> cd
myshell> ls /usr | wc -l
10
myshell> echo shellforge connects processes | tr a-z A-Z
SHELLFORGE CONNECTS PROCESSES
myshell> echo first > notes.txt
myshell> echo second >> notes.txt
myshell> sort -r < notes.txt
second
first
myshell> ls /missing 2> err.txt
myshell> cat err.txt
ls: cannot access '/missing': No such file or directory
myshell> nonexistent_command
ShellForge: nonexistent_command: No such file or directory
myshell> exit
Exiting ShellForge...

```

Figure 9: A ShellForge session: built-ins, a pipe, >, >>, <, 2> and an unknown command

```
ShellForge 10.0: demo-ipc

=====
Welcome to ShellForge Version 10.0
=====

myshell> demo-ipc

=====
ShellForge IPC Demonstration (CO-3: pipe() & IPC)
=====

[IPC Demo 1] Parent -> Child Unidirectional Communication
[Parent PID 1870] Sending to child: "Hello Child from Parent!"
[Child PID 1871] Received from parent: "Hello Child from Parent!"

[IPC Demo 2] Child -> Parent Unidirectional Communication
[Child PID 1872] Sending to parent: "Greetings Parent from Child!"
[Parent PID 1870] Received from child: "Greetings Parent from Child!"

[IPC Demo 3] EOF Detection Verification
Closing write-end of pipe (pipe3[1])...
Reader read() returned: 0 byte(s) (0 indicates EOF received)

=====
IPC Demonstration Completed Successfully
=====

myshell> exit
Exiting ShellForge...
```

Figure 10: demo-ipc: parent-to-child and child-to-parent messages, and end-of-file

c. *Memory*

A Valgrind session that touches every allocating path, including the IPC demo, the thread demo with four workers and the monitor thread, ends with all 31 heap blocks freed (15,921 bytes) and no errors (Figure 11). The joined threads leave nothing behind; with the handbook's detached monitor, Valgrind reports 272 bytes possibly lost and one error. GDB stops in `parse_line()`, shows `main()` as the caller, and confirms the vector ends in `NULL`.

```
Valgrind and GDB

$ valgrind --leak-check=full --show-leak-kinds=all ./bin/shellforge < session
(session: pwd, cd, ls, a pipe, > and <, 2>, demo-ipc, demo-threads, sleep with the monitor on)
HEAP SUMMARY:
  in use at exit: 0 bytes in 0 blocks
  total heap usage: 31 allocs, 31 frees, 15,921 bytes allocated
All heap blocks were freed -- no leaks are possible
ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

$ gdb -batch -ex 'break parse_line' -ex run -ex backtrace -ex 'print line' -ex finish ... ./bin/shellforge
Breakpoint 1, parse_line (line="echo gdb_probe") at src/parser.c:68
#0 parse_line (line=0x40a3d0 "echo gdb_probe") at src/parser.c:68
#1 0x000000004025f2 in main (argc=1, argv=0x7fffffffef8) at src/main.c:62
(gdb) print line          -> 0x40a3d0 "echo gdb_probe"
(gdb) print tokens[0]     -> 0x40a3d0 "echo"
(gdb) print tokens[1]     -> 0x40a3d5 "gdb_probe"
(gdb) print tokens[2]     -> 0x0
```

Figure 11: Valgrind: no leaks and no errors; GDB inspecting the argument vector

d. *Concurrency*

The race. With one thread nothing is lost, as expected. With more, the unprotected counter loses a growing share of its increments, and the amount varies from run to run, as a race does (Table 5). Two threads lost nothing in two of five runs, which is why a single clean run proves nothing. Under the mutex every run was exact, at 12 to 15 times the time once threads contend for the lock.

Table 5: Updates lost by an unprotected counter (100,000 increments per thread, five runs each)

Threads	Expected	Lost, mean (min–max)	Lost, %	Time without / with mutex
1	100,000	0 (0–0)	0.0	0.2 ms / 0.6 ms
2	200,000	10,271 (0–22,920)	5.1	0.2 ms / 3.1 ms
4	400,000	85,111 (5,686–140,582)	21.3	0.5 ms / 6.0 ms
8	800,000	331,881 (67,060–513,414)	41.5	0.9 ms / 13.8 ms
16	1,600,000	1,131,787 (958,086–1,321,295)	70.7	2.6 ms / 31.8 ms

Figure 12 shows the demonstration in the shell, with the monitor’s heartbeat arriving while the main thread waits for `sleep`. `/proc` lists two tasks with the monitor on and one with it off. ThreadSanitizer, built into the same shell, reports exactly one race: the unprotected `counter++` at `src/thread.c:109` in `unsafe_worker`. The mutex-protected worker and the monitor are clean (Figure 13).

```

ShellForge 10.0: demo-threads and the monitor thread (SHELLFORGE_MONITOR=1)

myshell> demo-threads 4 100000
[Threads] 4 threads x 100000 increments, expected total 400000
  Without a mutex: counter = 343658 (56342 updates lost to the race), 0.4 ms
  With a mutex: counter = 400000 (0 lost), 5.9 ms
[Threads] all 4 workers joined with pthread_join()
myshell> demo-threads 1 100000
[Threads] 1 thread x 100000 increments, expected total 100000
  Without a mutex: counter = 100000 (0 updates lost to the race), 0.2 ms
  With a mutex: counter = 100000 (0 lost), 0.6 ms
[Threads] the worker joined with pthread_join()
myshell> sleep 2.5

[Monitor] ShellForge Running...

[Monitor] ShellForge Running...
myshell> echo the shell kept answering
the shell kept answering
myshell> exit
Exiting ShellForge...

$ ls /proc/1874/task          # monitor on
1874
1877
$ ps -L -o pid,lwp,stat,comm -p 1874
PID  LWP  STAT  COMMAND
1874 1874  Sl    shellforge
1874 1877  Sl    shellforge
$ ls /proc/1886/task          # SHELLFORGE_MONITOR=0
1886

```

Figure 12: demo-threads and the monitor thread; the shell’s threads seen through `/proc`

```

ThreadSanitizer and the signal-mask mutation

$ gcc -g -fsanitize=thread -Iinclude src/*.c -o sf_tsan -pthread
$ printf 'demo-threads 2 20000\nexit\n' | SHELLFORGE_MONITOR=1 ./sf_tsan
WARNING: ThreadSanitizer: data race (pid=2347)
  #0 unsafe_worker src/thread.c:109 (sf_tsan+0x404f19)
  Previous write of size 8 at 0x000000409310 by thread T2:
  #0 unsafe_worker src/thread.c:109 (sf_tsan+0x404f2e)
SUMMARY: ThreadSanitizer: data race src/thread.c:109 in unsafe_worker

$ # mutation: the monitor thread created without blocking signals
300 commands, monitor thread running
with the mask: 0 stolen
without the mask (3 runs): 270 277 280
ShellForge: waitpid error: No child processes
ShellForge: waitpid error: No child processes
ShellForge: waitpid error: No child processes

```

Figure 13: ThreadSanitizer locating the race, and the signal-mask mutation

e. Mutation Tests

Each hazard fix was undone on purpose to confirm that the tests notice (Table 6, Figure 14).

Table 6: Mutation tests: undoing a fix must make its check fail

Fix undone	Effect	Caught by
Monitor thread created without blocking signals	270, 277 and 280 of 300 commands lost their exit status (0 with the fix)	test_week10.sh: 100 commands, no waitpid error
F_DUPFD instead of F_DUPFD_CLOEXEC	A child sees descriptors 10, 11 and 12 leaked from the shell	test_week9.sh: compares a child's descriptors with a plain ls
The handbook's detached monitor, never stopped	Valgrind: 272 bytes of thread storage possibly lost, 1 error	test_week10.sh: Valgrind "All heap blocks were freed"
A planted 200-byte leak; a heap overflow	Valgrind reports "definitely lost: 200 bytes"; ASan reports heap-buffer-overflow	test_week8.sh controls

```

Descriptor leak check (Week 9)

$ # correct build: saved descriptors are close-on-exec
myshell> ls /proc/self/fd > fds.txt
myshell> cat fds.txt
0
1
2
3
myshell> exit

$ # mutation: F_DUPFD instead of F_DUPFD_CLOEXEC
myshell> ls /proc/self/fd > fds.txt
myshell> cat fds.txt
0
1
10
11
12
2
3
myshell> exit

```

Figure 14: A redirected command's descriptors: correct build, and with F_DUPFD

f. The Web Page

The ShellForge page runs the real binary on the server. In Figure 15, four threads lose 416,836 of 800,000 increments without the mutex and none with it. Figure 16 shows the preset sessions and the transcript of the redirection session. On the backend image, sessions finish in 12–40 ms, except the monitor session, which waits 2.5 s by design. Eight threads with 200,000 increments each take 38 ms. Requests naming an unknown session, or asking for more than eight threads, are rejected with an explanation before anything runs.

Dashboard
Search
Repository
TextHack
ShellForge
ML insights
Administration

Search documents, ask a question, or enter keywords...
All categories

Demo Administrator

ShellForge

The OSSP Unix shell, written in C and running on this server. Each run starts the real program and shows what it printed.

Week 10: race condition and mutex

Each thread adds 1 to a shared counter. counter++ is a load, an add and a store, so two threads can load the same value and one increment is lost. Under a mutex only one thread is in that critical section at a time. Every worker is waited for with pthread_join(). The lost count changes from run to run.

Threads
Increments per thread

4

200000

Run the threads

Expected	800,000
Without a mutex	383,164 (416,836 lost) · 0.9 ms
With a mutex	800,000 (0 lost) · 13.6 ms

```

=====
Welcome to ShellForge Version 10.0
=====

myshell> demo-threads 4 200000
[Threads] 4 threads x 200000 increments, expected total 800000
Without a mutex: counter = 383164 (416836 updates lost to the race), 0.9 ms
With a mutex: counter = 800000 (0 lost), 13.6 ms
[Threads] all 4 workers joined with pthread_join()
myshell> exit
Exiting ShellForge...

```

Exit code 0 · 16 ms on the server

Figure 15: The race lab on the ShellForge page

Sessions

Each session types a fixed list of commands into the shell, in a fresh empty folder, with none of the server's own settings. Typing your own commands is not offered: it would let anyone run programs on the server.

WEEK 4
Processes: fork, exec and wait

Each external command runs in a child made by fork(), replaced by execvp() and collected by waitpid(). A missing program fails in the child, and the shell carries on.

```

echo Hello from a child
process
ls /usr/nonexistent_command
echo The shell is still here

```

Run

WEEK 5
Built-in commands

cd, pwd, env and help run inside the shell process: a child's chdir() would vanish with the child. env shows the bare environment the server gives the shell.

```

pwd
cd /usr/bin
pwd
cd
pwd
env
help

```

Run

WEEK 7
Pipes and IPC

pipe() and dup2() connect one command's output to the next one's input. demo-ipc passes messages between a parent and a child process and shows end-of-file.

```

echo shellforge connects
processes | tr s-z A-Z
ls /usr/bin | wc -l
demo-ipc

```

Run

WEEK 9
Redirection

open() and dup2() point a command's input, output or errors at a file: > truncates, >> appends, < reads and 2> captures errors.

```

echo first line > notes.txt
echo second line >> notes.txt
cat < notes.txt
sort -r < notes.txt > sorted.txt
cat sorted.txt
ls /missing 2> errors.txt
cat errors.txt
ls

```

Run

WEEK 10
Background monitor thread

A second thread in the shell process prints a heartbeat every second, while the main thread runs commands and waits for them.

```

echo The monitor thread reports every second
sleep 2.5
echo The monitor kept running while the main thread waited

```

Run

Redirection

```

=====
Welcome to ShellForge Version 10.0
=====

myshell> echo first line > notes.txt
myshell> echo second line >> notes.txt
myshell> cat < notes.txt
first line
second line
myshell> sort -r < notes.txt > sorted.txt
myshell> cat sorted.txt
second line
first line
myshell> ls /missing 2> errors.txt
myshell> cat errors.txt
ls: cannot access '/missing': No such file or directory
myshell> ls
errors.txt
notes.txt
sorted.txt
myshell> exit

```

Exit code 0 · 36 ms on the server

Figure 16: Preset sessions, and the transcript of the redirection session

g. Challenges

Most of the work went into the interactions between features and into the handbook's listings (Table 7).

Table 7: Challenges and how they were solved

Challenge	Solution
Week 3 listing does not compile (stray quote; Makefile never rebuilds)	Object-file Makefile, <code>-Iinclude</code> , <code>-Wall</code> <code>-Wextra</code> clean
Signal handlers call <code>printf()</code> , which is not async-signal-safe	Handlers use <code>write()</code>
SIGCHLD reaper steals the foreground command's exit status	Block SIGCHLD from <code>fork()</code> until the shell's <code>waitpid()</code> returns
A pipeline hangs: the reader never sees end-of-file	Close every unused pipe end in both children and the parent
Redirection listing: only <code>></code> and <code>>></code> , and not for built-ins	All four operators, applied in the shell around built-ins
Saved descriptors leak into programs the shell runs	<code>F_DUPFD_CLOEXEC</code> , proven by a mutation test
Handbook's monitor is detached and cannot be stopped	Condition-variable wait; <code>pthread_join()</code> at exit
A second thread receives SIGCHLD and steals exit statuses	Every new thread starts with all signals blocked
A web shell would let users run any command on the server	Preset sessions only; empty directory, bare environment, limits
ThreadSanitizer fails with 32-bit address randomisation	Run it under <code>setarch -R</code> ; report as skipped where that is blocked
CI runner: different checkout path and extra inherited descriptors	Compare with <code>\$(pwd)</code> and with a plain <code>ls</code> , not fixed values

5. CONCLUSION AND FUTURE WORK

ShellForge implements the shell the course describes, from reading a line to running threads, directly on POSIX system calls, and verifies each chapter with automated suites, sanitizers, Valgrind, ThreadSanitizer and the kernel's own view through `/proc`. The 180 passing checks, the leak-free Valgrind run and the measured race confirm the design. The two most instructive results were hazards rather than features: a reaper that steals exit statuses, and a thread that silently redirects signals. Both are present in the handbook's listings and are invisible in a quick manual test. Both are now fixed and guarded by tests that fail when the fix is removed. Through its web page, the same binary also serves as a safe, live demonstration inside the project's platform.

Future work.

- Job control: process groups per job, `&`, `jobs`, `fg` and `bg`, and handing the terminal over with `tcsetpgrp()`.
- Named pipes, Unix domain sockets and shared memory with semaphores, as a producer-consumer demonstration.
- Demonstrations of paging, page faults, `mmap` and copy-on-write after `fork()`.
- Pipelines of any length, and directory and inode operations for CO5.
- An interactive web terminal, only inside a disposable container per user with no network and no secrets.

REFERENCES

- [1] D. M. Ritchie and K. Thompson, "The UNIX time-sharing system," *Communications of the ACM*, vol. 17, no. 7, pp. 365–375, 1974.
- [2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley, 2013.
- [3] M. Kerrisk, *The Linux Programming Interface*. No Starch Press, 2010.
- [4] S. R. Bourne, "The UNIX shell," *The Bell System Technical Journal*, vol. 57, no. 6, pp. 1971–1990, 1978.
- [5] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Pearson, 2016.
- [6] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.
- [7] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018.
- [8] E. W. Dijkstra, "Solution of a problem in concurrent programming control," *Communications of the ACM*, vol. 8, no. 9, p. 569, 1965.
- [9] IEEE and The Open Group, *IEEE Std 1003.1-2017 (POSIX.1-2017): Standard for Information Technology—Portable Operating System Interface (POSIX) Base Specifications, Issue 7*, 2018.
- [10] D. R. Butenhof, *Programming with POSIX Threads*. Addison-Wesley, 1997.
- [11] N. Nethercote and J. Seward, "Valgrind: A framework for heavyweight dynamic binary instrumentation," in *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2007, pp. 89–100.

- [12] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A fast address sanity checker," in *Proc. USENIX Annual Technical Conference*, 2012, pp. 309–318.
- [13] K. Serebryany and T. Iskhodzhanov, "ThreadSanitizer: Data race detection in practice," in *Proc. Workshop on Binary Instrumentation and Applications (WBIA)*, 2009, pp. 62–71.
- [14] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.
- [15] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, 2014.